EXPLORATORY DATA ANALYSIS

Chapter 15

# Grouped Analysis: GroupBy, Crosstabs, and Pivot Tables

Comparing populations, summarizing patterns, and choosing the right denominator

**Course:** From Raw Data to Understanding

**Focus:** Grouped summaries • Aggregation • Contingency tables • Reshaping data

A practical chapter using Python, Pandas, NumPy, Matplotlib's object-oriented API, and Seaborn.

# Learning objectives

- Explain what a group represents and how grouping changes the unit of analysis.
- Use Pandas groupby to calculate descriptive summaries and multiple aggregations.
- Understand split–apply–combine and the difference between aggregation, transformation, and filtering.
- Construct, normalize, and interpret contingency tables with crosstab.
- Build pivot tables with multiple dimensions, values, and aggregation functions.
- Choose row, column, and overall denominators that match the analytical question.
- Handle missing groups, categorical ordering, small groups, and repeated observations responsibly.
- Communicate grouped findings without confusing descriptive differences with causal effects.

# How this chapter fits the course

The previous chapters examined individual variables, distributions, relationships, and dependence. Grouped analysis adds a practical way to compare subsets of observations. Instead of asking only 'What is the average order value?', we can ask 'How does average order value vary by customer segment, region, and month?'

## Review of earlier chapters (excluding Chapter 14)

- Chapters 1–3: define the observational unit, understand the data-generating process, inspect metadata, and assess completeness, validity, consistency, and uniqueness.
- Chapters 4–6: identify numerical, categorical, and temporal variables correctly; understand units, category levels, date parsing, and time granularity.
- Chapters 7–9: use robust descriptive statistics, quantiles, distribution summaries, skewness, and kurtosis rather than relying on a mean alone.
- Chapters 10–11: visualize distributions and pairwise relationships; inspect nonlinear patterns, overplotting, and subgroup differences.
- Chapter 12: compare multivariate patterns with facets, heatmaps, and small multiples, while making encodings, scales, and aggregation explicit.
- Chapter 13: select suitable association measures and interpret effect sizes, denominators, and the limits of correlation.

## Review of the immediately previous chapter: Chapter 14 — Covariance and Dependence

Covariance describes linear joint variation and depends on measurement units. Correlation standardizes covariance but still summarizes linear association. Independence is stronger than zero covariance: dependent variables can have zero covariance when nonlinear patterns cancel. Conditional distributions can differ in mean, variance, shape, or tail behavior.

## Chapter 14 assignment/project: expected-reasoning guide

- Changing a measurement unit scales covariance but does not change Pearson correlation under positive rescaling.
- A symmetric X and Y = $X^2$ example can be dependent while having zero covariance.
- Independence implies zero covariance when the relevant moments exist, but the reverse is not true.
- Conditional spread can change even when conditional means remain similar.
- Marginal distributions alone do not describe how observations are paired.

This is an expected-reasoning guide, not a grading of submitted work.

## Preview of this chapter

We will learn to group observations, calculate multiple summaries, attach group-level values back to rows, filter groups, construct contingency tables, and reshape data into pivot tables. Throughout, the central question is: what exactly is the denominator, and what does each row or cell represent?

# 1. Grouped analysis changes the question

A group is a subset of observations sharing one or more values or conditions. Examples include customers in the same region, transactions in the same month, or students in the same course section. Grouped analysis calculates summaries separately for these subsets.

> **Important:** grouping does not automatically create independent samples or comparable populations. It is an operation on the data, not a guarantee about the study design.

### 1.1 Observational unit versus group

Suppose a transaction dataset has one row per transaction. Grouping by customer produces customer-level summaries; grouping by month produces month-level summaries. These summaries have a different unit from the original transaction rows. Be explicit about that change.

### 1.2 A running example

We will use a small retail dataset with transactions, customer segments, regions, product categories, dates, quantities, and revenue. Each row represents one transaction line. The example is illustrative; real datasets require checking whether a row represents an order, an order line, a product event, or something else.

```
import pandas as pd
import numpy as np

sales = pd.DataFrame({
    "order_id": [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111,
            112],
    "region": ["North", "South", "North", "East", "South", "East",
               "North", "West", "West", "South", "East", "North"],
    "segment": ["Retail", "Business", "Retail", "Retail", "Business",
            "Retail",
                "Business", "Retail", "Business", "Retail", "Business",
                                "Retail"],
    "category": ["Books", "Tech", "Tech", "Books", "Books", "Tech",
                 "Books", "Tech", "Books", "Tech", "Books", "Tech"],
    "date": pd.to_datetime([
        "2026-01-03", "2026-01-04", "2026-01-05", "2026-01-08",
        "2026-01-12", "2026-01-15", "2026-02-02", "2026-02-04",
        "2026-02-07", "2026-02-10", "2026-02-15", "2026-02-18"
    ]),
    "quantity": [1, 2, 1, 3, 2, 1, 4, 2, 1, 3, 2, 1],
    "revenue": [20, 500, 35, 45, 60, 420, 80, 700, 55, 90, 75, 40]
})
sales["month"] = sales["date"].dt.to_period("M").astype(str)
```

# 2. The split–apply–combine pattern

Pandas groupby follows a conceptual three-step pattern:

- **Split:** divide observations into groups according to one or more keys.
- **Apply:** calculate a function for each group.
- **Combine:** assemble the group-level results into a Series or DataFrame.

The groups are defined by keys, while the applied operation determines the output. This pattern is the foundation for groupby, aggregation, transformation, and filtering.

```
# One grouping key
sales.groupby("region")["revenue"].mean()

# Multiple grouping keys
sales.groupby(["region", "segment"])["revenue"].mean()

# Explicitly choose output index behavior
sales.groupby("region", as_index=False)["revenue"].mean()
```

### 2.1 Single and multiple grouping keys

Grouping by one key creates one summary per distinct key value. Grouping by two keys creates one summary per observed combination. More keys create finer partitions, but group sizes usually become smaller. Excessive grouping can produce noisy summaries and sparse tables.

### 2.2 sort, dropna, observed, and as_index

- **sort:** controls whether group keys are sorted; it does not sort observations within each group.
- **dropna:** controls whether missing group keys are excluded. Make the policy explicit.
- **observed:** for categorical groupers, controls whether only observed category combinations are returned. Set it explicitly when output shape matters.

• **as_index:** controls whether group keys become the result index.

> Pandas defaults have changed across versions for some grouping behavior. For reproducible work, set relevant options explicitly and record the Pandas version.

# 3. Aggregation: summarize each group

Aggregation reduces each group to one or more summary values. Typical functions include count, size, sum, mean, median, standard deviation, minimum, maximum, and quantiles.

```
# One aggregation
region_mean = sales.groupby("region", as_index=False).agg(
    mean_revenue=("revenue", "mean")
)

# Several summaries of one variable
region_summary = sales.groupby("region").agg(
    revenue_mean=("revenue", "mean"),
    revenue_median=("revenue", "median"),
    revenue_sd=("revenue", "std"),
    revenue_min=("revenue", "min"),
    revenue_max=("revenue", "max"),
    transaction_count=("order_id", "count")
)

# Multiple variables and custom quantiles
segment_summary = sales.groupby("segment").agg(
    orders=("order_id", "nunique"),
    revenue_total=("revenue", "sum"),
    quantity_total=("quantity", "sum"),
    revenue_q90=("revenue", lambda s: s.quantile(0.90))
)
```

## 3.1 count versus size

`size` counts rows in each group, including rows where a selected column is missing. `count` counts non-missing values in a particular column. These can differ. If missing values exist, do not treat them as interchangeable.

```
g = sales.groupby("region")
row_counts = g.size()
revenue_nonmissing = g["revenue"].count()
unique_orders = g["order_id"].nunique()
```

## 3.2 Named aggregation

Named aggregation creates clear output column names and is usually easier to maintain than a MultiIndex of aggregation names. Prefer descriptive names that explain the statistic and unit, such as mean_revenue_bdt or median_delivery_days.

## 3.3 Custom aggregation

A custom function is useful when no built-in summary expresses the question. Keep it simple, document the definition, and consider whether a vectorized built-in method is available. Custom Python functions may be slower on large data.

```
def coefficient_of_variation(series):
    values = series.dropna()
    mean = values.mean()
    if len(values) < 2 or mean == 0:
        return np.nan
    return values.std(ddof=1) / abs(mean)

variation_by_region = sales.groupby("region").agg(
    revenue_cv=("revenue", coefficient_of_variation)
)
```

The coefficient of variation is only meaningful in suitable ratio-scale settings and can be unstable when the mean is near zero. Do not apply it mechanically to every variable.

# 4. Aggregation, transformation, and filtering

## 4.1 Aggregation

Aggregation returns a smaller result: typically one row per group. It answers questions such as 'What is the mean revenue in each region?'

## 4.2 Transformation

A transformation returns a result aligned to the original rows. It is useful when each observation needs a group-level reference value, such as the regional average or each row's deviation from its group mean.

```
sales["region_mean_revenue"] = (
    sales.groupby("region")["revenue"].transform("mean")
```

```
)
sales["difference_from_region_mean"] = (
    sales["revenue"] - sales["region_mean_revenue"]
)

# Within-group standardization (z-score)
sales["revenue_z_within_region"] = (
    sales["revenue"] -
    sales.groupby("region")["revenue"].transform("mean")
) / sales.groupby("region")["revenue"].transform("std")
```

A within-group z-score describes a value relative to its group's mean and standard deviation. It is not a causal adjustment and can be unstable in very small groups.

### 4.3 Filtering groups

Group filtering keeps or removes entire groups according to a condition. For example, retain regions with at least three transactions. This differs from row filtering, which selects individual observations.

```
regions_with_enough_rows = sales.groupby("region").filter(
    lambda group: len(group) >= 3
)

# Row-level filter using a group summary
sales["region_n"] = sales.groupby("region")["order_id"].transform("size")
large_groups_rows = sales.loc[sales["region_n"] >= 3].copy()
```

### 4.4 Which operation should you use?

| Operation | Typical output | Question |
|---|---|---|
| agg / aggregate | One row per group | What is the summary for each group? |
| transform | Same row alignment as input | What group-level value belongs on each row? |
| filter | Subset of original rows | Which entire groups meet a condition? |
| apply | Flexible, potentially varied | Can a custom group-wise operation not fit the above? |

> Use `apply` only when needed. It is flexible but can be slower and may produce complex or unpredictable output shapes. Prefer built-in groupby methods, aggregation, transformation, or filtering when they express the task.

## 5. Crosstabs: categorical relationships

A crosstab counts combinations of categorical values. It is a contingency table created from observations. It is especially useful for comparing category distributions across groups.

```
# Counts: segment by renewal status
customers = pd.DataFrame({
    "segment": ["Retail", "Retail", "Business", "Retail", "Business",
                "Business", "Retail", "Business", "Retail", "Retail"],
    "renewed": ["Yes", "No", "Yes", "Yes", "No",
                "Yes", "No", "Yes", "Yes", "No"]
})

counts = pd.crosstab(customers["segment"], customers["renewed"])
print(counts)
```

### 5.1 Counts and proportions answer different questions

Counts answer 'How many observations are in this cell?' Proportions answer 'What fraction of a chosen denominator is in this cell?' A proportion is incomplete unless its denominator is clear.

### 5.2 Normalize by rows, columns, or all observations

```
# Within each row: distribution of renewal status per segment
row_prop = pd.crosstab(
    customers["segment"], customers["renewed"], normalize="index"
)

# Within each column: composition of each renewal-status column
col_prop = pd.crosstab(
    customers["segment"], customers["renewed"], normalize="columns"
)

# Share of the entire dataset in each cell
overall_prop = pd.crosstab(
    customers["segment"], customers["renewed"], normalize="all"
)
```

- **Row normalization:** each row sums to 1; compare the outcome distribution within each row group.
- **Column normalization:** each column sums to 1; compare the composition of each column category.

- **Overall normalization:** all cells sum to 1; show each cell's share of the full dataset.

Example: if the question is 'What proportion of Retail customers renewed?', normalize by rows when segment is on rows. If the question is 'What proportion of all renewals came from Retail?', normalize by columns when renewal status is on columns.

### 5.3 Margins and totals

```
table_with_totals = pd.crosstab(
    customers["segment"],
    customers["renewed"],
    margins=True,
    margins_name="Total"
)
```

Margins add row and column totals. Totals are useful for context, but do not mistake them for additional independent observations.

### 5.4 Multiple categorical dimensions

```
pd.crosstab(
    [sales["region"], sales["segment"]],
    sales["category"],
    margins=True
)
```

Multi-index crosstabs can become difficult to read when there are many categories. Consider filtering to a meaningful subset, ordering categories, or presenting separate tables.

## 6. Pivot tables: flexible grouped summaries

A pivot table rearranges grouped summaries into a two-dimensional or multi-dimensional layout. In Pandas, `pivot_table` combines grouping, aggregation, and reshaping.

```
# Revenue sum by region and product category
revenue_pivot = pd.pivot_table(
    sales,
    values="revenue",
    index="region",
    columns="category",
    aggfunc="sum",
    fill_value=0,
    observed=True
)

# Multiple measures and aggregations
summary_pivot = pd.pivot_table(
    sales,
    values=["revenue", "quantity"],
    index="region",
    columns="segment",
    aggfunc={
        "revenue": ["sum", "mean"],
        "quantity": ["sum", "mean"]
    },
    observed=True
)
```

### 6.1 Index, columns, values, aggfunc

- **index:** variables defining the row groups.
- **columns:** variables spread across the columns.
- **values:** numerical variable or variables to summarize.
- **aggfunc:** the aggregation function or functions.

### 6.2 fill_value and missing combinations

A missing pivot cell can mean that no observations exist for that combination, or that the observed values are all missing, depending on the data and operation. Replacing missing cells with zero is a substantive choice, not just cosmetic formatting. Zero means a measured amount of zero; missing means unknown or unavailable.

### 6.3 margins and margins_name

```
pivot_with_totals = pd.pivot_table(
    sales,
    values="revenue",
    index="region",
    columns="category",
    aggfunc="sum",
    margins=True,
```

```
    margins_name="Grand Total",
    observed=True
)
```

### 6.4 pivot versus pivot_table

`DataFrame.pivot` reshapes data without aggregation and requires unique index/column combinations. `pivot_table` aggregates duplicate combinations using an aggregation function. If duplicate combinations exist, pivot_table is usually the appropriate summary tool.

### 6.5 Long and wide data

Long data generally stores one observation per row with variables in columns. Wide data spreads categories or time points across columns. Neither is universally better. Long format is often convenient for analysis and plotting; wide format can be convenient for reports. Reshaping changes layout, not the underlying information—unless aggregation is also performed.

## 7. Denominators, group size, and fair comparisons

A grouped mean or proportion is not automatically a fair comparison. Group sizes, variation, measurement definitions, and composition may differ.

### 7.1 Always show group size

```
group_report = sales.groupby("region", as_index=False).agg(
    n_rows=("order_id", "size"),
    revenue_total=("revenue", "sum"),
    revenue_mean=("revenue", "mean"),
    revenue_median=("revenue", "median")
)
```

A mean based on two observations is usually less stable than a mean based on two thousand. Show sample sizes and consider uncertainty or distribution plots when interpreting differences.

### 7.2 Sum versus mean

A larger total may simply reflect a larger group. A mean or per-unit rate can answer a different question. For example, total revenue describes volume; revenue per transaction describes average transaction value. Choose based on the business or scientific question.

### 7.3 Weighted averages

A grand mean of group means is generally not equal to the overall mean unless group sizes are equal or the group means happen to align. To combine group means correctly, weight each group mean by its number of valid observations (or the relevant exposure/weight).

```
# Verify overall mean from group means weighted by non-missing counts.
by_region = sales.groupby("region").agg(
    mean_revenue=("revenue", "mean"),
    n_valid=("revenue", "count")
)
weighted_mean = (
    by_region["mean_revenue"] * by_region["n_valid"]
).sum() / by_region["n_valid"].sum()

overall_mean = sales["revenue"].mean()
print(weighted_mean, overall_mean)
```

### 7.4 Rates need exposure denominators

For counts of events, a rate often needs an exposure denominator: incidents per 1,000 customers, defects per 10,000 units, or visits per person-year. Comparing raw counts across groups with different exposure can be misleading.

### 7.5 Simpson's paradox

A pooled group comparison can reverse or disappear after stratification because groups differ in their composition. This is not a mathematical error; it is a warning that the aggregation mixes different structures. Investigate relevant stratifiers and explain whether the pooled or within-group comparison answers the actual question. Stratification alone does not establish causality.

## 8. Ordering, missing groups, and categorical variables

- Use ordered categorical dtypes when a meaningful order exists, such as Low, Medium, High.

- Decide whether missing group keys should be excluded or retained as a group; document the policy.
- With categorical groupers, decide whether unobserved category combinations should appear.
- Do not interpret an absent combination as zero unless the data collection process confirms that it represents zero.
- Check for tiny groups, rare categories, and unstable statistics.
- Use explicit category labels and meaningful ordering in reports and plots.

```
priority_order = pd.CategoricalDtype(
    categories=["Low", "Medium", "High"],
    ordered=True
)
# Example if a priority column exists:
# tickets["priority"] = tickets["priority"].astype(priority_order)
# tickets.groupby("priority", observed=True).size()
```

# 9. Visualizing grouped summaries

Tables provide exact values; plots help reveal variation, overlap, and distribution shape. A mean-only bar chart can hide skewness, outliers, and group overlap.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Compare full distributions across regions.
fig, ax = plt.subplots(figsize=(8, 4.5))
sns.boxplot(data=sales, x="region", y="revenue", ax=ax)
ax.set_title("Revenue distribution by region")
ax.set_xlabel("Region")
ax.set_ylabel("Revenue")
fig.tight_layout()

# Plot grouped mean with the underlying observations visible.
fig, ax = plt.subplots(figsize=(8, 4.5))
sns.stripplot(
    data=sales, x="region", y="revenue",
    ax=ax, jitter=True, alpha=0.7
)
ax.set_title("Individual transaction revenue by region")
fig.tight_layout()
```

For larger datasets, use appropriate sampling, transparency, violin/box plots, or distribution summaries. Always make units and group sample sizes available.

# 10. A reusable grouped-summary function

```
def grouped_numeric_summary(
    data,
    group_columns,
    value_column
):
    required = list(group_columns) + [value_column]
    missing_columns = [c for c in required if c not in data.columns]
    if missing_columns:
        raise KeyError(f"Missing columns: {missing_columns}")

    result = (
        data.groupby(
            list(group_columns),
            dropna=False,
            observed=True,
            as_index=False
        )
        .agg(
            n_rows=(value_column, "size"),
            n_valid=(value_column, "count"),
            mean=(value_column, "mean"),
            median=(value_column, "median"),
            std=(value_column, "std"),
            minimum=(value_column, "min"),
            q25=(value_column, lambda s: s.quantile(0.25)),
            q75=(value_column, lambda s: s.quantile(0.75)),
            maximum=(value_column, "max")
        )
    )
    return result

region_category_summary = grouped_numeric_summary(
    sales,
    ["region", "category"],
    "revenue"
)
print(region_category_summary)
```

This helper is a starting point. For production use, consider configurable missing-key handling, type validation, custom quantiles, units, and a warning for very small groups.

# 11. Common mistakes

- Grouping at the wrong observational level, such as counting order lines when the question concerns orders.
- Confusing count with size when missing values exist.
- Reporting group means without group sizes or distributions.
- Comparing totals when group exposure differs.
- Normalizing a crosstab by the wrong dimension and answering a different question.
- Replacing every missing pivot cell with zero without checking what absence means.
- Assuming an unobserved category combination is a measured zero.
- Using apply for tasks that agg, transform, or filter can perform more clearly.
- Treating group differences as causal effects.
- Ignoring Simpson's paradox or subgroup composition.

# 12. Summary

- Groupby uses split–apply–combine to produce group-level results.
- Aggregation reduces groups; transformation returns row-aligned group information; filtering keeps or removes groups.
- Crosstabs summarize categorical combinations; normalization determines the denominator.
- Pivot tables reshape grouped summaries into a report-friendly layout.
- Group sizes, exposure, missingness, category definitions, and aggregation level affect interpretation.
- Means and totals answer different questions; weighted aggregation matters.
- Grouped comparisons are descriptive unless the study design supports causal interpretation.

# 13. Assignment

## Part A — Concepts

- Explain the difference between groupby aggregation, transformation, and filtering.
- Why can count and size return different results?
- Explain row, column, and overall normalization for a crosstab.
- Why is a missing pivot-table cell not automatically equivalent to zero?
- Why can the unweighted average of group means differ from the overall mean?

## Part B — Practical Pandas

Using the running sales dataset or a dataset of your choice:

- Calculate transaction count, total revenue, mean revenue, and median revenue by region.
- Repeat the analysis by region and segment.
- Add a group mean to every original row using transform and calculate each row's deviation.
- Build a crosstab of region by category with counts and row-normalized proportions.
- Build a pivot table of total revenue by region and category, then add totals.
- Identify the smallest group and explain why its summary may be unstable.

## Part C — Interpretation

A report says Region A has the highest total revenue, while Region B has the highest average revenue per transaction. Explain how both statements can be true. What additional summaries would you request before recommending a business decision?

### Part D — Simpson's paradox

Find or construct a dataset where a pooled comparison differs from within-group comparisons. Show the overall and stratified summaries, explain the composition difference, and state why neither table alone establishes causation.

## 14. Coding project — Grouped EDA Reporting Toolkit

**Goal:** Extend the cumulative EDA master notebook with a reusable grouped-analysis module suitable for a professional EDA report.

- Choose a dataset with at least two categorical grouping variables and two numerical measures.
- Document the observational unit and the meaning of each group.
- Create a group summary with counts, totals, means, medians, spread, and selected quantiles.
- Use transform to create at least one within-group comparison variable.
- Create at least two crosstabs, including one normalized in a way that directly answers a stated question.
- Create at least two pivot tables with clearly explained aggregation functions and missing-cell policy.
- Visualize at least one grouped distribution rather than showing only group means.
- Investigate a possible composition effect or Simpson's paradox.
- Write a concise interpretation that explains denominators, group sizes, and limitations.

**Deliverables:** a reproducible notebook section, labeled tables and figures, a short interpretation, and reusable functions. Keep the raw data intact; grouped summaries are analytical views, not replacements for the source observations.

## Preview of Next Chapter

**Chapter 16 — Outlier and Anomaly Detection**

We will develop a systematic approach to identifying unusual observations. Topics include univariate and multivariate outliers, IQR and robust-score methods, distance-based ideas, contextual anomalies, and the difference between flagging an observation and deciding what to do about it. The emphasis remains investigation—not automatic deletion.

## Selected references and further reading


- Pandas documentation: GroupBy user guide and split–apply–combine.
- Pandas API: DataFrame.groupby, DataFrame.agg, DataFrame.transform, DataFrame.filter.
- Pandas API: crosstab and pivot_table.
- McKinney, W. Python for Data Analysis. Chapters on data wrangling, group operations, and reshaping.
- Simpson, E. H. (1951). The interpretation of interaction in contingency tables. Journal of the Royal Statistical Society: Series B.
- Agresti, A. Statistical Methods for the Social Sciences and Categorical Data Analysis.


This chapter is educational material for exploratory data analysis. Grouped differences describe patterns in the observed data; causal conclusions require an appropriate design and additional assumptions.